Networks Overview

מערכת אוטונומית לפריסת Container מבוססת פרוטוקולי רשת

מגישים:

שקד חורש, דניאל בראנץ ונעה שהם

מטרת הפרוייקט

יצירת סוכן (Agent) המאפשר ללקוח רשת מזוהה לפרוס שרת Nginx אישי בתוך Docker, תוך מעבר דרך כל שכבות המודל

הדמיית תהליך הגישה של לקוח לשירות רשת:

לקוח => DHCP – הקצאת כתובת IP ללקוח => DNS – תרגום שם שרת => שרת אפליקציה => פריסת Container באמצעות Docker

הסבר על הפרויקט

המערכת שבנינו מורכבת ממספר שרתים שמדמים את המרכיבים המרכזיים בתקשורת אינטרנט מודרנית:

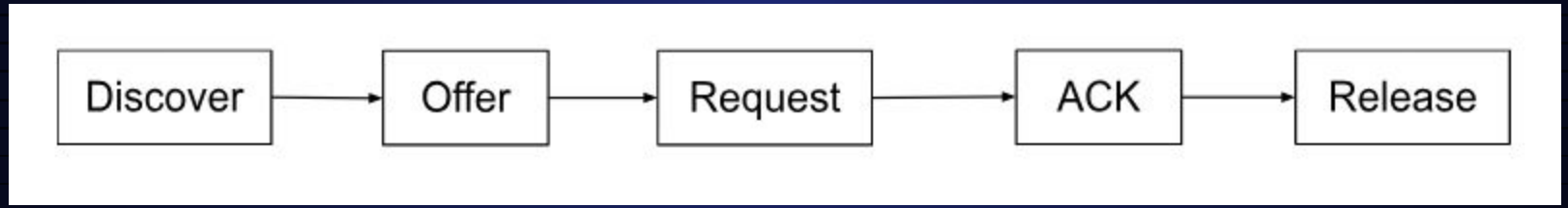
1. שרת DHCP – מקצה ללקוח כתובת IP ופרטי רשת.
2. שרת DNS מקומי – מתרגם שם דומיין לכתובת IP.
3. שרת אפליקציה – מקבל בקשות מהלקוח ומפעיל Agent שמרים Container באמצעות Docker.

שרת ה-DHCP

תפקידו של שרת ה DHCP:

חלוקת כתובות IP ופרטי רשת (Gateway, Subnet) באופן דינמי

תרשים זרימה:



תהליך ה-DORA בקוד:

- **Discover:** הלקוח פונה לשרת המוכר לקבלת IP.
- **Offer:** השרת מציע IP פנוי מתוך ה-**IP Pool** (טווח 10.0.0.2-20).
- **Request:** הלקוח מאשר את ה-IP המוצע (שימוש ב-XID לסנכרון).
- **ACK:** השרת רושם את ה-**Lease** (השכרה) במאגר ומאשר.

שחרור כתובת (Release) אוטומטי נעשה בסיום העבודה לחיסכון במשאבים.

שרת ה-DNS

תפקידו של שרת ה DNS:

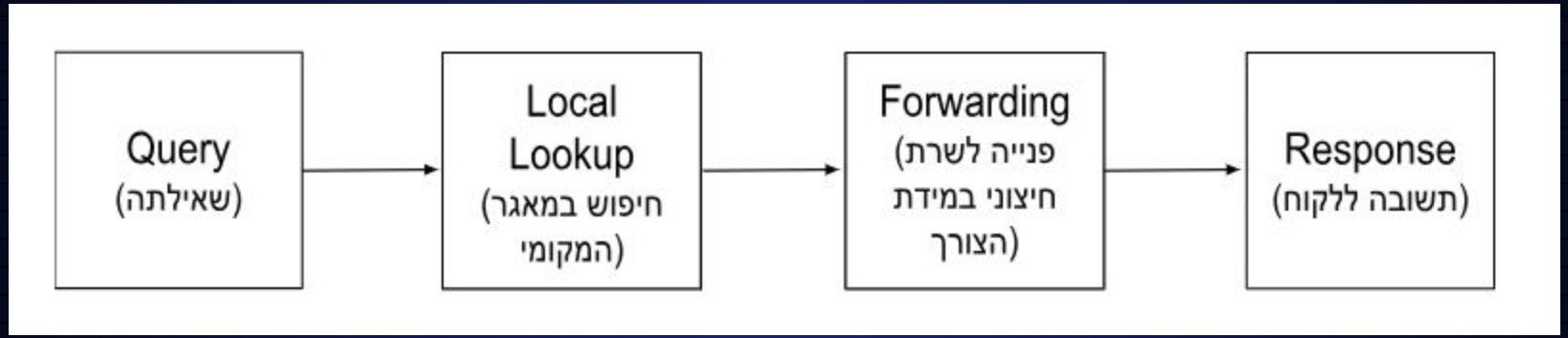
תרגום שמות דומיין (myagent.local) לכתובות IP.

מימוש טכני:

- **Binary Extraction:** פענוח חבילות UDP בינאריות בפורמט DNS תקני.
- **Local Records:** מאגר מקומי המפנה את הלקוח לשרת האפליקציה.
- **DNS Forwarding:** יכולת פנייה לשרת חיצוני (8.8.8.8) עבור כתובות אינטרנט ציבוריות.

תוצאה: הלקוח לא צריך לדעת את ה-IP של השרת, רק את שמו.

תרשים זרימה:


Query
(שאילתה)
Local
Lookup
(חיפוש במאגר
המקומי)
Forwarding
(פנייה לשרת
חיצוני במידת
הצורך)
Response
(תשובה ללקוח)

שרת האפליקציה
ופרוטוקול RUDP

ארכיטקטורה של שרת האפליקציה:

שרת רב־תהליכי (Multithreaded) המטפל ב-TCP ו-RUDP במקביל.

תפקיד שרת האפליקציה:

- קבלת בקשת הלקוח
- עיבוד הנתונים
- פריסת Container באמצעות Docker
- החזרת תשובה ללקוח

RUDP – Reliable UDP:

מימוש אמינות מעל UDP.

מנגנונים עיקריים:

- **Handshake:** יצירת חיבור מסודר (SIN -> SIN_ACK -> ACK).
- **Sliding Window:** שליחת מספר חבילות במקביל לשיפור ביצועים.
- **Congestion Control:** הגדלת חלון בהצלחה והקטנתו ב-Timeout (שידור חוזר).

תקשורת: העברת Payload בפורמט JSON הכולל את פרטי ה-Container.

האפליקציה -
Agent מבוסס Docker

Docker Agent:

תהליך העבודה:

- קבלת נתוני הלקוח
- יצירת דף HTML מתוך Template
- ניקוי Container קיים באותו פורט
- הרצת Container חדש עם Nginx

תוצאה: שרת אתר אישי עבור הלקוח.

הזרקת נתונים:

קבלת template.html ומילוי פרטי הלקוח בזמן אמת.

אוטומציה:

שימוש ב-subprocess להרצת פקודות Docker.

מנגנון Cleanup:

מניעת התנגשויות על ידי סריקה וכיבוי קונטיינרים קודמים באותו פורט (kill_container_on_port).

תוצאה סופית: שרת Nginx חי ופעיל בפורט המבוקש.

ארכיטקטורת הלקוח (Client)

תפקיד הלקוח:

תרגום שמות דומיין (myagent.local) לכתובות IP.

תהליך העבודה של הלקוח:

1. קבלת IP משרת DHCP
2. תרגום שם דומיין באמצעות DNS
3. בחירת פרוטוקול תקשורת (TCP / RUDP)
4. שליחת בקשת פריסה לשרת האפליקציה
5. הקמת Container
6. שחרור כתובת IP

מימוש טכני:

- Binary Extraction: פענוח חבילות UDP בינאריות בפורמט DNS תקני.
- Local Records: מאגר מקומי המפנה את הלקוח לשרת האפליקציה.
- DNS Forwarding: יכולת פנייה לשרת חיצוני (8.8.8.8) עבור כתובות אינטרנט ציבוריות.

תוצאה: הלקוח לא צריך לדעת את ה-IP של השרת, רק את שמו.

סיכום - המערכת מדגימה:

- הקצאת כתובות באמצעות DHCP
- תרגום שמות באמצעות DNS
- תקשורת TCP ו-RUDP
- מנגנוני אמינות בתקשורת
- פריסה אוטומטית של שירות באמצעות Docker

תודה על ההקשבה!